

Unravel-RDF: A JavaScript Framework for Building Web Applications over RDF Knowledge Graphs*

Olivier Filangi^{1,*}, Nils Paulhe², Guillaume Laisney³, Clément Frainay⁴ and Franck Giacomoni⁵

¹*Institute for Genetics, Environment and Plant Protection (IGEPP), National Research Institute for Agriculture, Food and Environment (INRAE), Université Rennes, Institut Agro, Le Rheu, France*

⁴*Vrije Universiteit Amsterdam, De Boelelaan 1105, 1081 HV Amsterdam, The Netherlands*

⁵*University of Skövde, Höskolevägen 1, 541 28 Skövde, Sweden*

Abstract

Semantic Web technologies are widely used to integrate heterogeneous life science resources as RDF knowledge graphs exposed via SPARQL endpoints. While these graphs support rich querying, exploiting them in interactive web applications remains difficult for frontend developers. Existing tools focus on data access and query execution, but provide limited support for application-oriented abstractions such as composable query builders or domain-specific languages tightly aligned with UI logic.

We present Unravel-RDF, a JavaScript framework that introduces a focus-based domain-specific language (DSL) for constructing SPARQL queries together with a composable API tailored to event-driven user interfaces. Unravel-RDF offers functional abstractions for incremental query construction, asynchronous execution, pagination, and caching, while delegating query processing to existing SPARQL engines. We evaluate Unravel-RDF in the FORVM ecosystem, which interconnects biomedical and plant knowledge graphs through shared chemical entities, and demonstrate a web application for cross-domain exploration of metabolomics relationships across multiple SPARQL endpoints. This case study suggests that Unravel-RDF can bring knowledge graph querying closer to modern frontend development practices and improve the modularity and responsiveness of metabolomics exploration tools.

Keywords

Knowledge graphs, JavaScript framework, Frontend development, Domain-specific language (DSL), Metabolomics,

1. Introduction

Semantic Web technologies are widely used to represent and query knowledge in the life sciences.[1] Many resources are published as RDF knowledge graphs and exposed via SPARQL endpoints, enabling interoperability between heterogeneous datasets describing molecular entities, biological processes, pathways, and phenotypic characteristics across diverse organisms.[2]. In bioinformatics, these approaches support the integration and reuse of these resources, but their effective use within analysis platforms and web applications still poses significant technical challenges.

Developers building web frontends on top of RDF knowledge graphs face a fundamental challenge: bridging low-level RDF graph models with application logic. Frontend developers typically either embed raw SPARQL queries directly in their code or rely on low-level RDF/JS-based libraries that expose graph primitives and query execution but offer limited support for structuring application data access.

The RDF/JS ecosystem, including engines such as Comunica, has been highly successful at standardizing RDF representations and query interfaces across heterogeneous data sources. However, these tools are intentionally designed as foundational building blocks, and do not provide the kind of higher-level, application-oriented abstractions—such as composable query builders, domain-specific languages (DSLs), or data mappers—that frontend developers routinely rely on in relational database ecosystems.

ISWC 2026

*Corresponding author.

✉ olivier.filangi@inrae.fr (O. Filangi)

ORCID: 0000-0002-2094-3271 (O. Filangi); 0000-0003-4550-1258 (N. Paulhe); 0009-0005-5653-3257 (G. Laisney); 0000-0003-4313-2786 (C. Frainay); 0000-0001-6063-4214 (F. Giacomoni)



© 2022 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

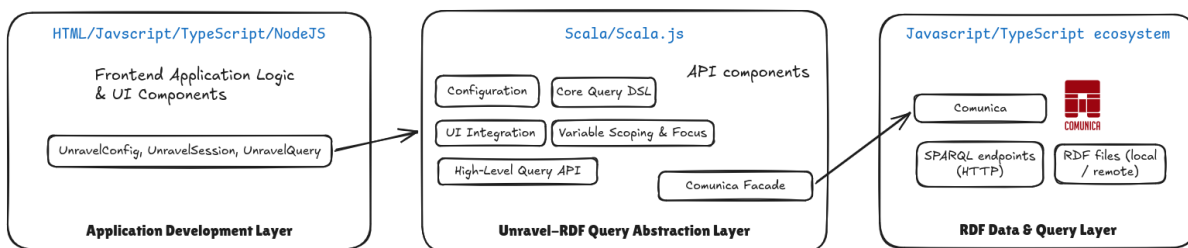


Figure 1: Unravel-RDF architecture: a layered design connecting RDF data sources, the Unravel query abstraction layer, and UI integration components.

As a result, there remains a significant gap between RDF query formulation and modern frontend development practices, leading to brittle integrations and weak alignment between knowledge graph queries and application-level structures.

To address this gap, we introduce Unravel-RDF, a JavaScript framework that combines a DSL for SPARQL query construction with a composable API tailored to event-driven user interface applications. Our design provides functional abstractions for incremental, focus-based query building, asynchronous execution, pagination, and caching over RDF data sources. We evaluate Unravel-RDF on the FORVM ecosystem, which interconnects biomedical and plant knowledge graphs through shared chemical entities, and demonstrate a cross-domain exploration workflow implemented as a web application that leverages these abstractions to support interactive navigation of metabolomics relationships across multiple SPARQL endpoints.

2. Architecture and Programming Model

2.1. Layered Architecture

Figure 1 illustrates the architecture of Unravel-RDF as a three-layer system: an *Application Development Layer*, an *Unravel Query Abstraction Layer*, and an *RDF Data & Query Layer*. The Application Development Layer hosts JavaScript/TypeScript code, HTML-based user interfaces, and Node.js services, together with Unravel sessions that are embedded into web applications. This layer includes hooks and UI components (e.g., built with D3.js, React, or Vue) that consume Unravel queries, as well as three immutable objects, `UnravelConfiguration`, `UnravelSession`, and `UnravelQuery`, which encapsulate configuration, query construction, and execution respectively.

The Unravel Query Abstraction Layer is implemented in Scala/Scala.js and provides the core query model. It is organized into several functional subsystems: a configuration module, a core query DSL, a high-level query API, UI integration utilities, and a variable scoping mechanism. Together, these components offer a composable, focus-based abstraction over SPARQL queries that can be safely reused across application code. A dedicated `Comunica.js` facade connects this abstraction layer to the JavaScript RDF/JS ecosystem and exposes a typed interface to the underlying SPARQL engines.

Finally, the RDF Data & Query Layer corresponds to the existing JavaScript/TypeScript ecosystem for RDF and SPARQL. In our deployment, this layer is instantiated by a `Comunica` SPARQL engine that can query over multiple RDF sources, including HTTP-accessible SPARQL endpoints and local or remote RDF files. Unravel-RDF therefore focuses on bridging application-level query construction and UI logic with these heterogeneous RDF data sources, without reimplementing query processing.

2.2. Programming Model

At its core, Unravel-RDF provides a domain-specific language (DSL) for constructing SPARQL queries. Instead of relying on raw query strings, the DSL enables the composition of structured query fragments. Query construction is incremental and follows a lambda-closure style, where functions capture context

Table 1

Core query building primitives in Unravel-RDF

Primitive	Role	Description
<code>something(name, f)</code>	Focus introduction	Introduces a fresh SPARQL variable as focus; <code>f</code> builds patterns relative to it.
<code>from(name, f)</code>	Focus rebinding	Reuses an existing variable as focus; <code>f</code> continues query construction from it.
<code>out(prop, obj, f)</code>	Forward expansion	Adds <code><focus> prop obj</code> and sets focus to <code>obj</code> before <code>f</code> .
<code>in(prop, subj, f)</code>	Backward expansion	Adds <code>subj prop <focus></code> and sets focus to <code>subj</code> before <code>f</code> .
<code>traverse(prop, node, f)</code>	Bidirectional	Adds <code>{<focus> prop node} UNION {node prop <focus>}</code> and sets focus to <code>node</code> .

Figure 2: Incremental query construction with Unravel-RDF and resulting SPARQL

<pre>const session = UnravelSession(config) .something("compound", compound => compound.out("?rel", "CHEBI:24279")); const session2 = session.from("compound", compound => compound.out("rdf:type", "?type"));</pre>	<pre>SELECT ?compound ?rel ?type WHERE { ?compound ?rel CHEBI:24279 . ?compound rdf:type ?type . }</pre>
---	--

and enable modular composition. All manipulated objects are immutable, enforcing a functional programming paradigm that improves the composability of query transformations.

Queries are constructed relative to a focus variable, which serves as an implicit context for navigating RDF graphs. Focus-binding primitives (`something`, `from`) introduce or rebind this variable, defining the scope of query construction. In contrast, graph-expansion primitives (`out`, `in`, `traverse`) add triple patterns relative to the current focus while updating it to the newly reached node. These primitives accept both RDF IRIs and SPARQL variables as arguments, so the same combinators can express schema-level templates and data-level queries in a uniform way. Table 1 summarizes the core primitives and their semantics, and the following example shows how they can be composed to build a query centered on a compound entity acting as the focus variable. In this example², `session2` extends an initial query over the focus variable `compound`, illustrating how Unravel-RDF supports incremental refinement of SPARQL queries. In practice, these primitives are exposed through `UnravelSession` objects, which provide a fluent API for constructing and reusing focus-based queries.

2.3. Query Execution

Unravel-RDF does not reimplement SPARQL query processing. Instead, it delegates query evaluation to existing engines through the `Comunica.js` facade presented in Figure 1. This facade is implemented in `Scala.js` and exposes a typed interface that maps `UnravelQuery` objects to RDF/JS-compatible data sources and SPARQL endpoints. As a result, Unravel-RDF can execute queries over heterogeneous RDF data sources while reusing the standard RDF and SPARQL tooling available in the JavaScript/TypeScript ecosystem.

This integration enables access to multiple RDF data sources and supports federation mechanisms, allowing queries to transparently combine distributed datasets. The system therefore focuses on bridging high-level query composition and application logic with existing execution infrastructures, while leaving optimization, query planning, and low-level data access to the underlying SPARQL engines.

2.4. User Interface Integration

On the application side, Unravel-RDF is designed to integrate with modern, event-driven user interfaces. The execution model is based on JavaScript promises combined with a subscription mechanism, enabling asynchronous and staged processing of queries. Results are delivered in batches through a pagination mechanism and exposed as lazy pages (windows of results) that are only fetched when requested, which avoids long-running SPARQL requests and reduces perceived latency.

These abstractions are intended to be consumed directly from hook-based and component-based UI frameworks such as React, Vue, or D3-powered visualizations. Application code can subscribe to `UnravelQuery` objects, trigger incremental loading of result pages, and update views as data becomes available, following typical reactive programming patterns. Additionally, a caching layer prevents redundant query execution by reusing previously retrieved results, thereby improving performance and responsiveness. Overall, this integration brings SPARQL-based querying closer to modern frontend development practices while preserving a clear separation between UI concerns, query abstraction, and engine-level execution.

2.5. Implementation and Distribution

Unravel-RDF is implemented in functional Scala and compiled to JavaScript using `Scala.js`, which allows the same core query abstraction layer to be reused across browser-based and Node.js applications. It is distributed through standard web channels, including CDN delivery and the npm ecosystem, facilitating integration into modern JavaScript and TypeScript build pipelines. The source code and documentation are publicly available in the project repository¹, enabling reuse and extension by third-party developers.

3. Case Study: FORVM

We evaluate Unravel-RDF in the context of the FORVM ecosystem, which provides a cross-domain knowledge graph infrastructure spanning biomedical and plant sciences. The biomedical component, *FORVM ChemDisease*, is a curated knowledge graph derived from statistical co-occurrence of MeSH-annotated chemical and disease concepts in PubMed literature, while *FORVM Plants* extends this approach to plant sciences by leveraging Planteome ontologies to represent plant-related entities and traits.²

These two knowledge graphs are interconnected through shared chemical entities, enabling cross-domain exploration of metabolomics relationships across multiple SPARQL endpoints, including FORVM ChemDisease and FORVM Plants, as well as RDF Portal endpoints for resources such as ChEBI, PubChem, and NCBI.³ We implement this workflow as a web application built with Unravel-RDF, in which the interface displays clusters of nodes linking chemical compounds to both MeSH disease terms and Planteome concepts, revealing transitive connections between biomedical and plant knowledge.

In this application, Unravel-RDF orchestrates compositional query construction and reuse of focus-based query fragments while issuing separate SPARQL queries to the different endpoints as needed. The resulting query logic is encapsulated in reusable Unravel-RDF sessions and queries, so that the UI can trigger navigation steps (e.g., expanding a compound to related diseases or traits, then following links to external knowledge graphs) without embedding raw SPARQL strings in the frontend code.

Acknowledgments

We thank the MetaboHUB national infrastructure (funded by the French National Research Agency under the France 2030 programme) for its support in developing the metabolomics data management and analysis ecosystem underlying the FORVM knowledge graph and the Unravel-RDF application

¹<https://forge.inrae.fr/p2m2/unravel-rdf>

²The FORVM use case UI is available at <http://kg.p2m2.inrae.fr/>.

³For example, ChEBI is exposed at <https://rdfportal.org/ebi/sparql> and PubChem RDF at <https://rdfportal.org/pubchem/sparql>.

layer. We also acknowledge the INRAE metaprogram Digit-Bio for funding positions and projects that contributed to the design and implementation of the FORVM ecosystem and, ultimately, to the development of Unravel-RDF.

Declaration on Generative AI

During the preparation of this work, the authors used Perplexity and ChatGPT in order to: paraphrase and reword text, improve writing style and clarity, and perform grammar and spelling checks. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the publication's content.

References

- [1] H. Wu, A. Yamaguchi, Semantic Web technologies for the big data in life sciences, *Bioscience Trends* 8 (2014) 192–201. doi:10.5582/bst.2014.01048.
- [2] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg, G. Appleton, M. Axton, A. Baak, N. Blomberg, J.-W. Boiten, L. B. da Silva Santos, P. E. Bourne, J. Bouwman, A. J. Brookes, T. Clark, M. Crosas, I. Dillo, O. Dumon, S. Edmunds, C. T. Evelo, R. Finkers, A. Gonzalez-Beltran, A. J. G. Gray, P. Groth, C. Goble, J. S. Grethe, J. Heringa, P. A. C. 't Hoen, R. Hooft, T. Kuhn, R. Kok, J. Kok, S. J. Lusher, M. E. Martone, A. Mons, A. L. Packer, B. Persson, P. Rocca-Serra, M. Roos, R. van Schaik, S.-A. Sansone, E. Schultes, T. Sengstag, T. Slater, G. Strawn, M. A. Swertz, M. Thompson, J. van der Lei, E. van Mulligen, J. Velterop, A. Waagmeester, P. Wittenburg, K. Wolstencroft, J. Zhao, B. Mons, The FAIR Guiding Principles for scientific data management and stewardship, *Scientific Data* 3 (2016) 160018. URL: <https://www.nature.com/articles/sdata201618>. doi:10.1038/sdata.2016.18, number: 1.

A. Online Resources

The following resources provide additional material related to the FORVM ecosystem and the Unravel-RDF programming model:

- Unravel-RDF source code and documentation: <https://forge.inrae.fr/p2m2/unravel-rdf>
- FORVM DiseasesChem : <https://forum-webapp.semantic-metabolomics.fr>
- Supplementary examples and usage tutorials: <https://example.org/unravel-examples>